Phase 9 SOP – Deployments, Scaling, Rolling Updates, Rollbacks & HPA (EKS)

Objective

In this phase we move beyond simply running containers.

Until now, we learned:

- Pods
- Deployments
- Services
- Ingress
- ConfigMaps
- Secrets
- Persistent Volumes

Now we will learn how Kubernetes manages application lifecycle in production.

By the end of this phase you will understand:

✅ Manual Scaling

✅ Self-Healing

✅ Rolling Updates

✅ Rollbacks

✅ Metrics Server

✅ Horizontal Pod Autoscaler (HPA)

✅ Automatic Scale Up

✅ Automatic Scale Down

Real World Scenario

Imagine an e-commerce application.

Normal day: `3 Pods`

Black Friday Sale: `10 Pods`

Traffic Drops: `3 Pods`

Application Upgrade: `v1 → v2`

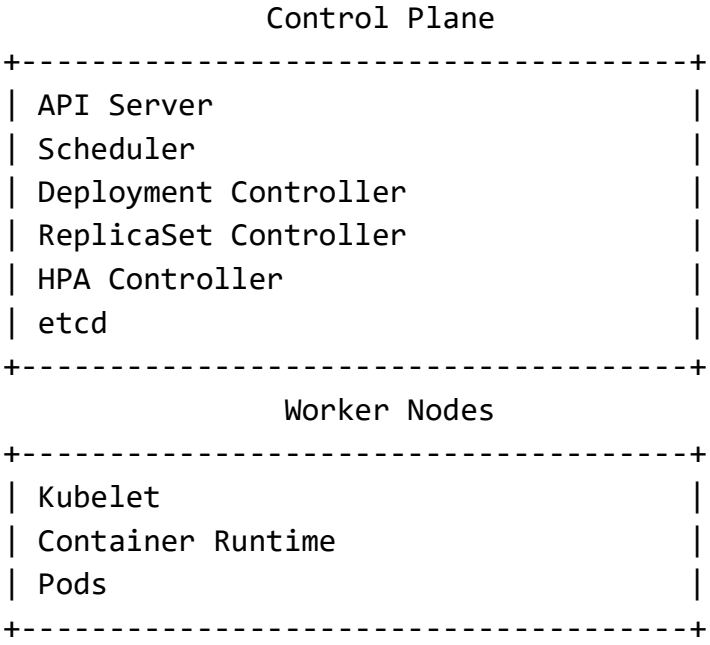
Bug Found: `v2 → v1`

Node Crash: `Dead Pod → New Pod`

All of this is handled automatically by Kubernetes.

That is exactly what we will learn in Phase 9.

Architecture Overview



Phase 9A – Manual Scaling

Objective: Learn how Kubernetes increases or decreases pod count.

Verify Current Deployment:

```
kubectl get deployment
```

Example:

```
NAME               READY   UP-TO-DATE   AVAILABLE
nginx-deployment   1/1     1            1
```

Check Deployment Details

```
kubectl describe deployment nginx-deployment
```

Observe: `Replicas: 1 desired`

Scale Deployment

Increase replicas:

```
kubectl scale deployment nginx-deployment --replicas=3
```

Output:

```
deployment.apps/nginx-deployment scaled
```

Verify

```
kubectl get deployment
```

Output:

```
NAME               READY
nginx-deployment   3/3
```

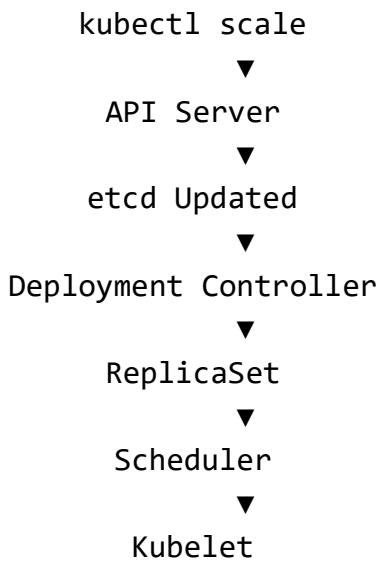
Watch Pod Creation

```
kubectl get pods -w
```

Example:

```
nginx-deployment-xxxx Running
nginx-deployment-yyyy Running
nginx-deployment-zzzz Running
```

What Happened Internally?



▼

New Pods Running

Verify Node Placement

```
kubectl get pods -o wide
```

Example:

```
NAME                        NODE
nginx-deployment-xxx        Node-1
nginx-deployment-yyy        Node-1
nginx-deployment-zzz        Node-2
```

Scheduler decides placement.

Phase 9B – Self Healing

Objective

Understand how Kubernetes recreates failed pods.

Delete a Pod

Choose one pod:

```
kubectl delete pod nginx-deployment-xxxxx
```

Watch Pods

```
kubectl get pods -w
```

Example:

```
nginx-deployment-xxxxx    Terminating

nginx-deployment-yyyyy    Pending

nginx-deployment-yyyyy    Running
```

What Happened?

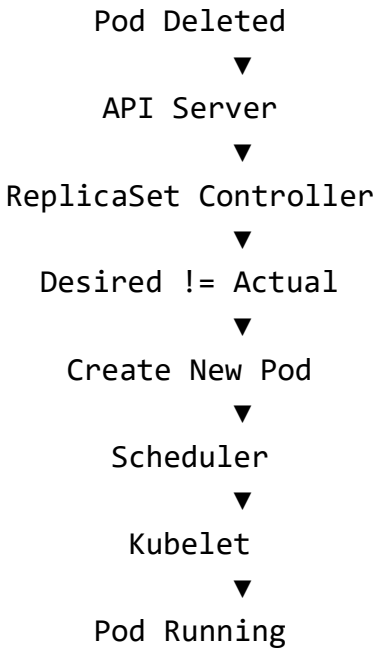
ReplicaSet noticed:

```
Desired Pods = 3

Actual Pods = 2
```

ReplicaSet immediately created a replacement pod.

Internal Flow



Phase 9C – Rolling Updates

Objective

Update applications without downtime.

Check Current ReplicaSets

```
kubectl get rs
```

Example:

```
nginx-deployment-b6485fcbb
```

Update Image

Open a second terminal:

```
kubectl get pods -w
```

Update image:

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.29.1
```

Output:

```
deployment.apps/nginx-deployment image updated
```

Observe Rolling Update

Example:

```
Old Pod Terminating

New Pod Pending

New Pod Running

Old Pod Terminating

New Pod Running
```

One pod at a time.

No downtime.

Verify ReplicaSets

```
kubectl get rs
```

Example:

```
OLD RS → 0 replicas

NEW RS → 3 replicas
```

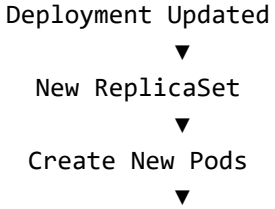
What Happened?

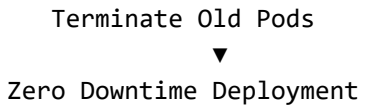
Deployment Controller created:

```
New ReplicaSet
```

and gradually moved traffic.

Internal Flow



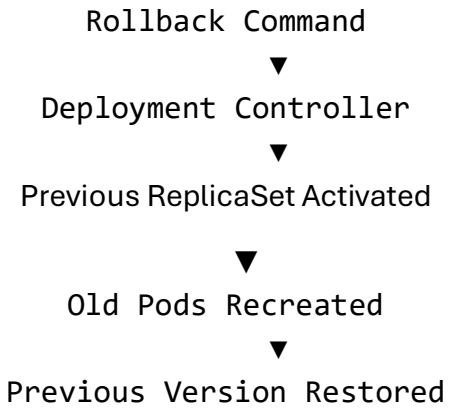


Phase 9D – Rollback

Objective

Restore previous working version.

Internal Flow



View Rollout History

```
kubectl rollout history deployment nginx-deployment
```

Example:

```
REVISION
1
2
```

Rollback

```
kubectl rollout undo deployment nginx-deployment
```

Output:

```
deployment.apps/nginx-deployment rolled back
```

Verify ReplicaSets

```
kubectl get rs
```

Example:

```
Old ReplicaSet → 3 replicas

New ReplicaSet → 0 replicas
```

Observe Pods

```
kubectl get pods -w
```

You will see:

```
Old Version Pods Created

New Version Pods Removed
```

Phase 9E – Horizontal Pod Autoscaler (HPA)

Objective

Automatically scale pods based on CPU utilization.

Why HPA?

Without HPA:

```
Traffic ↑

Still 3 Pods
```

Application becomes slow.

With HPA:

```
Traffic ↑

Pods ↑
```

Automatically.

Step 1 – Install Metrics Server

Verify:

```
kubectl top nodes
```

If you see: `Metrics API not available`

Install Metrics Server:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Verify

```
kubectl get pods -n kube-system
```

Example: `metrics-server-xxxx Running`

Verify Metrics

```
kubectl top nodes
```

Example:

```
NODE                                        CPU    MEMORY

ip-172-31-17-46                             1%     9%
ip-172-31-40-167                            1%     9%
```

Step 2 – Add Resource Requests

Edit deployment YAML.

Add:

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"

  limits:
    cpu: "200m"
    memory: "256Mi"
```

Why?

HPA calculates:

```
CPU Utilization

Current Usage
-------------
Requested CPU
```

Without requests:

```
cpu: <unknown>
```

Apply:

```
kubectl apply -f nginx-deployment.yml
```

Step 3 – Create HPA

Create autoscaler:

```
kubectl autoscale deployment nginx-deployment `
  --cpu-percent=50 `
  --min=2 `
  --max=6
```

Note: `--cpu-percent,` is deprecated but still works.

Verify

```
kubectl get hpa
```

Example:

```
NAME               TARGETS

nginx-deployment   cpu: 1%/50%
```

Step 4 – Generate Traffic

Open Terminal 1:

```
kubectl get hpa -w
```

Open Terminal 2:

```
kubectl get pods -w
```

Open Terminal 3:

```
kubectl run load-generator --rm -it --image=busybox -- /bin/sh
```

Inside container:

```
while true; do wget -q -O- http://nginx-service; done
```

Observe Scale Down

Initial: `Replicas = 3`

`CPU = 29%`

`Target = 50%`

HPA decided: `Too many pods`

Scaled: `3 → 2`

Verification:

`kubectl get hpa`

Example:

`cpu: 29%/50%`

`REPLICAS = 2`

Observe Scale Up

Delete existing HPA:

`kubectl delete hpa nginx-deployment`

Create new HPA:

```
kubectl autoscale deployment nginx-deployment `
  --cpu-percent=25 `
  --min=2 `
  --max=6
```

Now HPA sees:

`CPU = 27%`

`Target = 25%`

Decision:

`Need More Pods`

Result:

2 → 3

Verification

```
kubectl get hpa
```

Example:

```
NAME               TARGETS

nginx-deployment   cpu: 27%/25%
```

Pods:

```
kubectl get pods
```

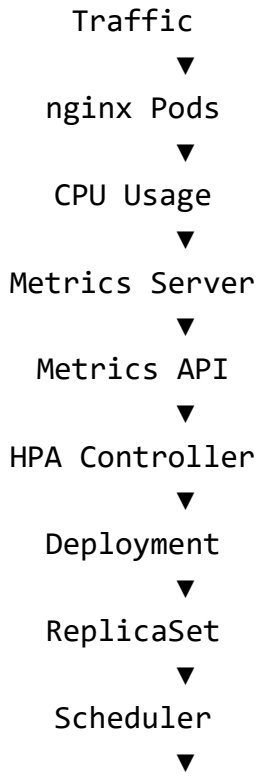
Example:

```
nginx-deployment-xxxxx

nginx-deployment-yyyyy

nginx-deployment-zzzzz
```

HPA Internal Flow



```
Kubelet
   ▼
More Pods
```

Key Learning Outcomes

By completing Phase 9 you have successfully implemented:

- ✅ Manual Scaling
- ✅ Scheduler Placement
- ✅ ReplicaSet Reconciliation
- ✅ Self Healing
- ✅ Rolling Updates
- ✅ Rollbacks
- ✅ Metrics Server
- ✅ Resource Requests & Limits
- ✅ Horizontal Pod Autoscaler
- ✅ Automatic Scale Down
- ✅ Automatic Scale Up

Production Takeaway

HPA scales:

```
Pods
```

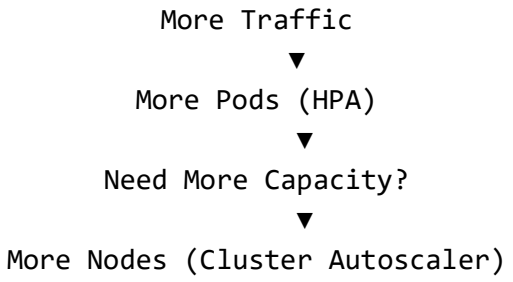
Node Autoscaling scales:

```
Worker Nodes
```

Real production EKS clusters typically run:

```
HPA
+
Cluster Autoscaler
```

Together:


More Traffic
▼
More Pods (HPA)
▼
Need More Capacity?
▼
More Nodes (Cluster Autoscaler)